Code Execution Engine Documentation

Lohit P Talavar, Adarsh Mishra, Priyashi Meena

September 7, 2025

Project Overview

This project is a simple and robust code execution engine built with Flask and Docker. It provides a secure environment to execute code in different languages with resource constraints (time and memory limits).

Features

- **Secure Execution:** Code is executed in an isolated Docker container to prevent any harm to the host system.
- **Language Support:** Supports C, C++, and Python.
- **Resource Limits:** Enforces time and memory limits on code execution.
- **Simple API:** A single API endpoint to submit code for execution.
- **Comprehensive Test Suite:** Includes unit and API tests to ensure reliability.

API Endpoint

‘/api/execute‘

This is the main endpoint to submit code for execution.

- **Method:** ‘POST‘
- **Content-Type:** ‘application/json‘

Request Payload

Field	Type	Required	Description
‘language‘	string	Yes	The programming language (‘c‘, ‘c++‘, ‘python‘).
‘code‘	string	Yes	The source code to execute.
‘stdin‘	string	No	Standard input for the code. Defaults to empty.
‘timelimit‘	integer	No	Time limit in seconds. Defaults to 2.
‘memorylimit‘	integer	No	Memory limit in megabytes. Defaults to 1024.

Example Request:

```
{
    "language": "python",
    "code": "print('Hello, World!')",
    "stdin": "",
    "timelimit": 5,
    "memorylimit": 128
}
```

Listing 1: Example Request

Response Payload

Field	Type	Description
‘success‘	boolean	‘true‘ if the code executed successfully, ‘false‘ otherwise.
‘stdout‘	string	The standard output of the code.
‘stderr‘	string	The standard error of the code.
‘err‘	string	A high-level error message (e.g., ”Compilation Error”, ”Time Limit Exceeded”).
‘timetaken‘	float	The time taken for the code to execute in seconds.
‘memorytaken‘	float	The memory consumed by the code in megabytes.

Example Response (Success):

```
{
    "success": true,
    "stdout": "Hello, World!\n",
    "stderr": "",
    "err": "",
    "timetaken": 0.01,
    "memorytaken": 7.6
}
```

Listing 2: Example Success Response

Example Response (Error):

```
{
    "success": false,
    "stdout": "",
    "stderr": "Traceback (most recent call last):\n  File \"/
       sandbox/temp/solution.py\", line 1, in <module>\n    1 /
        0\nZeroDivisionError: division to zero\nCommand exited
       with non-zero status 1",
    "err": "Runtime Error (Exit Code: 1)",
    "timetaken": 0.02,
    "memorytaken": 7.5
}
```

Listing 3: Example Error Response

Supported Languages

- C (‘c‘)
- C++ (‘c++‘)
- Python (‘python‘)

Example Response (Error):

```
{
    "success": false,
    "stdout": "",
```

```
    "stderr": "Traceback (most recent call last):\n  File \"/sandbox/temp/solution.py\", line 1, in <module>\n    1 / 0\nZeroDivisionError: division to zero\nCommand exited with non-zero status 1",
    "err": "Runtime Error (Exit Code: 1)",
    "timetaken": 0.02,
    "memorytaken": 7.5
}
```

Listing 4: Example Error Response

How to Interact with the API

This section provides examples of how to interact with the '/api/execute' endpoint using various tools and programming languages.

Using cURL:

```
curl -X POST -H "Content-Type: application/json" \
     -d "{\"language\": \"python\", \"code\": \"print('Hello from cURL!')\"}" \
     http://127.0.0.1:5000/api/execute
```

Listing 5: cURL Example

Using Python 'requests' library:

```
import requests
import json

url = "http://127.0.0.1:5000/api/execute"
headers = {"Content-Type": "application/json"}
payload = {
    "language": "python",
    "code": "print('Hello from Python requests!')"
}

response = requests.post(url, headers=headers, data=json.dumps(payload))
print(response.json())
```

Listing 6: Python Requests Example

Using JavaScript (Fetch API):

```
const url = "http://127.0.0.1:5000/api/execute";
const payload = {
    language: "python",
    code: "print('Hello from JavaScript Fetch!')"
};

fetch(url, {
    method: "POST",
    headers: {
        "Content-Type": "application/json"
    },
    body: JSON.stringify(payload)
})
.then(response => response.json())
.then(data => console.log(data))
```

```
.catch(error => console.error('Error:', error));
```

Listing 7: JavaScript Fetch Example

Using React with Axios:

```
import React, { useState } from 'react';
import axios from 'axios';

function CodeExecutor() {
    const [output, setOutput] = useState(null);

    const executeCode = async () => {
        const payload = {
            language: "python",
            code: "print('Hello from React with Axios!')"
        };
        try {
            const response = await axios.post('http
                ://127.0.0.1:5000/api/execute', payload);
            setOutput(response.data);
        } catch (error) {
            console.error('Error executing code:', error);
            setOutput({ error: error.message });
        }
    };

    return (
        <div>
            <button onClick={executeCode}>Execute Code</button>
            {output && <pre>{JSON.stringify(output, null, 2)}</
                pre>}
        </div>
    );
}

export default CodeExecutor;
```

Listing 8: React with Axios Example

Using React without Axios (Fetch API):

```
import React, { useState } from 'react';

function CodeExecutor() {
    const [output, setOutput] = useState(null);

    const executeCode = async () => {
        const payload = {
            language: "python",
            code: "print('Hello from React without Axios!')"
        };
        try {
            const response = await fetch('http
                ://127.0.0.1:5000/api/execute', {
                method: 'POST',
                headers: {
                    'Content-Type': 'application/json',
                },
                body: JSON.stringify(payload),
```

```
            });
            const data = await response.json();
            setOutput(data);
        } catch (error) {
            console.error('Error executing code:', error);
            setOutput({ error: error.message });
        }
    };

    return (
        <div>
            <button onClick={executeCode}>Execute Code</button>
            {output && <pre>{JSON.stringify(output, null, 2)}</
                pre>}
        </div>
    );
}

export default CodeExecutor;
```

Listing 9: React without Axios Example

Supported Languages

- C (‘c‘)
- C++ (‘c++‘)
- Python (‘python‘)

How to Run

1. Installation

First, make sure you have Docker installed and running.

Then, install the required Python packages using pip:

```
# Make sure you are in the project root directory
# and you have activated your virtual environment

pip install -r requirements.txt
```

Listing 10: Installation Command

2. Running the Application

To start the Flask application, run the following command:

```
python app.py
```

Listing 11: Run Application Command

The server will start on ‘http://127.0.0.1:5000‘.

Core Python Files

This section outlines the main Python files that constitute the Code Execution Engine project.

‘app.py‘

This is the main Flask application file. It sets up the ‘/api/execute‘ endpoint, handles incoming JSON requests, extracts code execution parameters (‘code‘, ‘language‘, ‘stdin‘, ‘timelimit‘, ‘memorylimit‘), and then calls the ‘execute_code‘ function from ‘execute.py‘. It also configures CORS (Cross-Origin Resource Sharing) and returns the execution result as a JSON response.

‘execute.py‘

This file contains the core logic for secure code execution within Docker containers. The ‘execute_code‘ function performs the following key operations:

- Validates the programming language provided.
- Checks for Docker availability and builds a ‘code-runner-image‘ if it doesn't exist.
- Uses temporary directories on the host to store user code and input files.
- Starts a Docker container in detached mode.
- Copies the code and input files into the container.
- Compiles C/C++ code if the language requires it.
- Executes the code within the Docker container, enforcing specified time and memory limits using ‘timeout‘ and ‘/usr/bin/time‘.
- Parses the execution output, including standard output (‘stdout‘), standard error (‘stderr‘), time taken (‘timetaken‘), and memory consumed (‘memorytaken‘).
- Determines the success or failure of the execution, identifying issues like Time Limit Exceeded, Memory Limit Exceeded, Runtime Errors, or Compilation Errors.
- Ensures proper cleanup of the Docker container after execution.

‘run_single_test.py‘

This is a utility script primarily used for development and debugging. It allows developers to run a specific unit test (‘$\text{test}_fibonacci_performance‘from‘TestCodeExecution‘in‘tests.test_execution‘)withoutexecutingtheentiretestsuit$

‘tests/$\text{test}_api.py$‘

This file contains integration tests for the Flask API. It uses the ‘unittest‘ framework to:

- Start and stop the Flask development server as a background process before and after tests, respectively.
- Send POST requests to the ‘/api/execute‘ endpoint.
- Assert the HTTP status code and the content of the JSON responses.
- Cover various scenarios, including successful executions, compilation errors, and performance tests across different supported languages (Python, C, C++).

‘tests/test$_e$*xecution.py*‘

This file contains unit tests for the ‘execute_code‘ function, directly testing the core code execution logic without involving the Flask API. It uses the ‘unittest‘ framework and includes a ‘run_test‘ helper function for detailed output. The tests cover a comprehensive range of scenarios, including:

- Basic ”Hello World” executions for Python, C, and C++.
- Handling of multi-line standard input.
- Time Limit Exceeded (TLE) scenarios for all supported languages.
- Memory Limit Exceeded (MLE) scenarios for all supported languages.
- Various Runtime Errors, such as division by zero and segmentation faults.
- Compilation Errors.
- Performance-related tests (e.g., Fibonacci sequence calculation).

How to Run Tests

The project comes with a comprehensive test suite to ensure everything is working correctly. For detailed information on running and understanding the tests, please refer to the Tests README.

Tests

This directory contains the test suite for the Code Execution Engine. The tests are divided into two main categories:

1. Unit Tests

- **File:** ‘test_execution.py‘
- **Description:** These tests directly invoke the ‘execute_code‘ function from ‘execute.py‘. They are designed to verify the core logic of the code execution engine, including language support, resource limits, and error handling, without involving the Flask API.
- **How to Run:**

```
python -m unittest tests/test_execution.py
```

Listing 12: Run Unit Tests

2. API Tests

- **File:** ‘test_api.py‘
- **Description:** These tests interact with the Flask API endpoint (‘/api/execute‘). They verify the end-to-end functionality of the system, including the Flask application, request/response handling, and integration with the code execution engine. The API tests automatically start and stop the Flask development server.
- **How to Run:**

```
python -m unittest tests/test_api.py
```

Listing 13: Run API Tests

Credits and Contributions

This project was developed by Lohit P Talavar.

Authors

We welcome contributions! If you'd like to contribute, please refer to our 'CONTRIBUTING.md' (if available) or open an issue/pull request on the GitHub repository.

Here are some of the key Authors to this project:

- Lohit P Talavar
- Adarsh MIshra
- Priyanshi Meena

How to Get the Repository

To get a local copy of this project, you can clone the repository using Git:

```
git clone https://github.com/lohitpt252003/judge-image-for-
    annaforces.git
cd judge-image-for-annaforces
```

Listing 14: Clone Repository